

PROMPTING FUNDAMENTALS ON REAL TASKS v2

Prompt Iteration Log · RagBucket Technical Case Study

Task

Real FL-01 task: Use AI to pressure-test a real project, expose weak claims, and turn project evidence into clear portfolio writing. The project used here is RagBucket.

Baseline — Naive Prompt

Prompt

Write a case study for my RagBucket project.

Observed output excerpt: “I can’t write this yet — I don’t know anything about RagBucket beyond the name. If I write it now, I’ll just generate generic RAG-project boilerplate...” The model then asked about the problem, architecture, what I built versus glued together, failures, evaluation, and limitations.

Note: The prompt gave only a project name. The model correctly avoided fabrication, but it produced questions instead of a case study. Next: assign a critical technical role.

Version 1 — Role Assignment

Prompt

You are a critical technical portfolio reviewer.

Write a case study for my RagBucket project.

Output observation: The response became more skeptical and focused on defensible technical claims. It still could not write the case because the project facts were missing.

What changed and why: I added a critical reviewer role so the model would challenge weak claims instead of writing marketing copy.

Observed difference: Sharper questions, but still no usable case. Next: add context and motivation.

Version 2 — Context and Motivation

Prompt

You are a critical technical portfolio reviewer.

I am building an AI/ML engineering portfolio for small startup teams looking for practical open-source solutions. RagBucket started from a question: traditional ML models can be saved and moved, but a RAG retrieval setup is usually tied to the project that built it. I explored whether retrieval memory and configuration could be packaged into a transferable artifact.

Write a case study for RagBucket.

Output observation: The answer now framed the project around portability and transferability instead of generic “better retrieval.” The story became project-specific, but “portable” could still be overstated.

What changed and why: I added the real project motivation and audience.

Observed difference: Better relevance and specificity. Next: add a style example that keeps limitations visible.

Version 3 — Few-Shot Example

Prompt

Keep the previous role and context.

Use this style as an example:

“RagBucket packages a FAISS index, chunks, and retrieval configuration into one .rag artifact. It works for the local FAISS case, but I have not yet benchmarked portability across environments.”

Write plainly. Keep limitations visible. Now draft the RagBucket case study.

Output observation: The writing became shorter, more technical, and less promotional. The model followed the honesty pattern and kept the FAISS limitation visible.

What changed and why: I added one example of the evidence level and tone I wanted.

Observed difference: Voice improved, but decisions and results could still blur together. Next: force an output structure.

Version 4 — Output Structure

Prompt

Keep the previous role, context, and style.

Write using exactly these sections:

1. The Problem
2. What I Did and Decided
3. What Came of It
4. What I'd Do Differently

Do not invent metrics or production claims. Only use facts I provide.

Output observation: The case became easier to scan. The portability problem stayed separate from design decisions such as zipfile packaging, FAISS, manifest-based embedding configuration, and LLM decoupling. Missing benchmarks stayed in the result section.

What changed and why: I added an exact four-part structure.

Observed difference: Organization improved significantly. A one-shot prompt still depended on the facts I remembered to include. Next: decompose the task into an interview-first workflow.

Version 5 — Step Decomposition

Prompt

You are a critical technical portfolio reviewer.

I want to turn one real AI/ML project into an honest technical case study for small startup teams looking for practical open-source AI/ML solutions.

Work in steps:

1. Interview me one question at a time about the real problem.
2. Ask what I actually built and the important design decisions.
3. Ask about trade-offs, failed ideas, and limitations.
4. Ask for real results and evidence. Challenge claims I cannot support.
5. Ask what I would do differently.
6. Only after the interview, write the case study.

Use: The Problem; What I Did and Decided; What Came of It; What I'd Do Differently.

Voice: direct, curious, honest, technical, plain, no buzzwords.

Do not invent benchmarks. Do not exaggerate success. Do not claim production readiness. Only use facts I provide.

After drafting, list every place where you had to guess.

Observed result: This was the strongest workflow. The interview exposed a real reframe: I started thinking portability meant putting the whole RAG system into one file. Building RagBucket made me think portability is better framed as making the retrieval pipeline reproducible and transferable, even when external retrieval state cannot live inside the artifact.

What changed and why: I decomposed the task into evidence collection, claim checking, reflection, and drafting.

Observed difference: The model became an interviewer and reviewer before becoming a writer. This produced the most specific and defensible case.

Cross-Model Comparison: Claude vs ChatGPT

Area	Claude	ChatGPT
Tone	More skeptical and interviewer-like. It pushed for concrete failures and unsupported claims.	More instructional and solution-oriented. It tends to organize next steps quickly.
Claim checking	Strong at refusing missing facts and challenging portability and benchmark claims.	Strong with explicit constraints, but can move toward drafting sooner.
Structure	Natural long-form interview and critique.	Clear procedural structure and concise formatting.
Failure point	Can keep questioning and delay the draft.	Can polish too early unless told to interview first.

Comparison: Claude was more useful for pressure-testing the project story and forcing missing evidence into the open. ChatGPT was more useful for turning a defined process into a clear reusable workflow. The interview-first instructions and factual constraints matter on both models.

Final Reusable Prompt Template

You are a critical technical portfolio reviewer.

I want to turn one real project into an honest technical case study for **{audience}**.

The project is: **{project name and one-line description}**.

Work in steps:

1. Interview me one question at a time about the real problem or motivation.
2. Ask what I actually built and the important decisions I made.
3. Ask about trade-offs, failed approaches, and known limitations.
4. Ask for measured results or real evidence. Challenge any claim I cannot support.
5. Ask what I would do differently now.
6. Only after the interview, write the case study.

Use these sections:

- The Problem
- What I Did and Decided
- What Came of It
- What I'd Do Differently

Voice: **{voice card}**.

Do not invent metrics, users, benchmarks, failures, or production claims. Only use facts I provide. Keep limitations visible. After drafting, list every place where you had to guess so I can correct it.

Final Reflection

Role assignment improved skepticism. Context made the answer specific. The example improved voice. Output structure improved readability. The biggest improvement came from step decomposition: interview first, test the claims, then write. That process surfaced real weaknesses in my own project story instead of hiding them behind polished AI copy.

Verification note

The baseline excerpt and RagBucket interview observations come from my documented prompt-development workflow. The cross-model section should be checked against saved runs of the exact Version 5 prompt in Claude and ChatGPT before final submission.